

1 Fibonacci series

Write a program that prompts the user to enter a positive integer 'n.' Your program should then generate and display the first 'n' numbers of the Fibonacci series. For example, if the user enters '5,' the program should output: **0,1,1,2,3,.** If n is not a positive number then print **Invalid Input.**

Sample Input 1:

5

Sample Output 1:

0,1,1,2,3,

Sample Input 2:

-5

Sample Output 2:

Invalid Input

Input	Output	Score
4	0,1,1,2,	0
7	0,1,1,2,3,5,8,	10
-1	Invalid Input	10
11	0,1,1,2,3,5,8,13,21,34,55,	10
3	0,1,1,	10
20	0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,1597,2584,4181,	10

2 Smallest Prime Number

Scenario:

In a small village fair, children participate in a game where they select a number of two different ranges. The game master, with enthusiasm, announces that a special prize awaits those who choose a prime number. Excitement fills the air as the children eagerly search for their lucky prime picks. The program needs to efficiently use a for loop to iterate through possible prime numbers and determine the smallest 5 primes greater than the user's input.

Task:

Craft a program that prompts a user to input an integer, find the smallest 5 prime number greater than a given input, and output the result. *(The statements inside brackets are for your understanding purpose only)*

Sample Input :

10 (Input)

Sample Output :

11(Output of five prime numbers)

13

17

19

23

Input	Output	Score
99	101 103 107 109 113	0
20	23 29 31 37 41	10
46	47 53 59 61 67	10
132	137 139 149 151 157	10
75	79 83 89 97	10

	101	
16	17 19 23 29 31	10

3 Prime or Composite number

Scenario :

As the program runs, it prompts Alex to input a number, and magically, the code reveals whether the number is a prime or composite, unlocking the hidden knowledge within the ancient book. Excited, Alex shares the program with friends, turning an ordinary day into an extraordinary adventure of code-breaking and discovery.

Task :

Craft a program that prompts the user to input a number. The program should then determine whether the entered number is a prime or composite number.

(The statements inside brackets are for your understanding purpose only)

SAMPLE INPUT 1:

7 (input)

SAMPLE OUTPUT 1:

7 is a prime number

SAMPLE INPUT 2:

12 (input)

SAMPLE OUTPUT 2:

12 is a composite number

Input	Output	Score
2	2 is a prime number	0
15	15 is a composite number	10
50	50 is a composite number	10
29	29 is a prime number	10
97	97 is a prime number	10
121	121 is a composite number	10

4 Series Sum Calculator

A software developer working on a project for a mathematics education platform. The platform aims to provide students with interactive tools to explore and understand various mathematical concepts. Your current task is to develop a program that calculates the sum of a series based on user input. The series is defined by taking the next term and the number of terms from the user. The program should be user-friendly, efficient, and capable of handling different types of series.

Task:

Craft a program that prompts the user to input the next term of a series and the number of terms in the series. The program then calculates the sum of the series and displays it along with the series itself.

(The statements inside brackets are for your understanding purpose only)

Sample Input 1:

3 (Next term)
4 (Number of terms in series)

Sample Output 1:

3 + 33 + 333 + 3333 (The series)
3702 (sum of the series)

Sample Input 2:

5
3

Sample Output 2:

5 + 55 + 555
615

Input	Output	Score
2 5	2 + 22 + 222 + 2222 + 22222 24690	0
1 6	1 + 11 + 111 + 1111 + 11111 + 111111 123456	10
9 2	9 + 99 108	10
7 4	7 + 77 + 777 + 7777 8638	10
6 4	6 + 66 + 666 + 6666 7404	10
4 3	4 + 44 + 444 492	10

5 Divisor Sum and Equality Checker

Imagine you have a mysterious code that takes an integer input from the user, finds its positive divisors, calculates their sum, and finally checks if the sum is equal to the original number. This code has a secret power to identify whether the number entered by the user is equal or not based on the sum of its divisors.

Task:

Craft a program to prompt the user to input a number. A loop is used to find and display the positive divisors of the entered number. If a number i is a divisor, it is printed, and its value is added to the sum. After finding and displaying all the divisors, it prints the sum of the divisors. Finally, it checks if the sum is equal to the original number and prints a output

eg: X is an equal number

Y is not an equal number

(The statements inside brackets are for your understanding purpose only)

Sample Input 1:

6

Sample Output 1:

1 2 3 (positive divisors)

6 (sum of the divisors)

6 is an equal number

Sample Input 2:

42

Sample Output2:

1 2 3 6 7 14 21

54

42 is not an equal number

Input	Output	Score
28	1 2 4 7 14 28 28 is an equal number	0
25	1 5 6 25 is not an equal number	10
8128	1 2 4 8 16 32 64 127 254 508 1016 2032 4064 8128 8128 is an equal number	10
496	1 2 4 8 16 31 62 124 248 496 496 is an equal number	10
16	1 2 4 8 15	10

	16 is not an equal number	
78	1 2 3 6 13 26 39 90 78 is not an equal number	10

6 Abundant Number

SCENARIO:

In the quiet town of Mathville, where numbers roamed freely and equations danced in the air, there lived a young programmer named Alex. Alex had a knack for solving numerical puzzles and unravelling the mysteries hidden within the digits. One sunny afternoon, as Alex sat in front of their computer, a riddle whispered through the digital winds. It spoke of abundant numbers, those elusive integers whose divisors danced in a sum greater than the numbers themselves. Intrigued, Alex set out on a quest to craft a program that could decipher the abundant numbers from the ordinary.

Task:

Craft a program that prompts the user to enter the input. An abundant number is a positive integer for which the sum of its proper divisors (excluding itself) is greater than the number itself. Your program should be able to take an integer input from the user and check whether a given integer number is an abundant number or not.

abundant number :

(i.e) 12

$$1+2+3+4+6=16$$

$$16>12$$

12 is Abundant number

(The statements inside brackets are for your understanding purpose only)

Sample Input 1:

12 (Input)

Sample Output 1:

12 is an Abundant number (Output)

Sample Input 2:

7 (Input)

Sample Output 2:

7 is not an Abundant number (Output)

Input	Output	Score
18	18 is an Abundant number	0
13	13 is not an Abundant number	10
24	24 is an Abundant number	10
16	16 is not an Abundant number	10
30	30 is an Abundant number	10
25	25 is not an Abundant number	10

7 Counted the number of leap and non-leap years

SCENARIO:

Once upon a time, in a quaint little town, there was a community of curious individuals who loved exploring the mysteries of time. In this town, a group of friends was fascinated by the concept of leap years, those magical extra days that appear every four years. Determined to understand more about leap years, they decided to create a program that not only identified whether a given year was a leap year but also counted the number of leap and non-leap years in the upcoming decade.

Task:

The friends gathered around their computers, determined to create a program that would take a user-inputted year, determine if it was a leap year, and then count the number of leap and non-leap years in the next decade.

(The statements inside brackets are for your understanding purpose only)

Sample Input 1:

2022 (Input-year)

Sample Output 1:

2022 is not a Leap Year.

Leap Years: 3 (Count of Leap years in the next decade)

Non-Leap Years: 7(Count of non-Leap years in the next decade)

Sample Input 2:

2020 (Input-year)

Sample Output 2:

2020 is a Leap Year.

Leap Years: 2

Non-Leap Years: 8

Input	Output	Score
2024	2024 is a Leap Year. Leap Years: 2 Non-Leap Years: 8	0
2010	2010 is not a Leap Year. Leap Years: 3 Non-Leap Years: 7	10
2100	2100 is not a Leap Year. Leap Years: 2 Non-Leap Years: 8	10
1996	1996 is a Leap Year. Leap Years: 2 Non-Leap Years: 8	10
2018	2018 is not a Leap Year. Leap Years: 3 Non-Leap Years: 7	10

1984	1984 is a Leap Year. Leap Years: 2 Non-Leap Years: 8	10
------	--	----

8 Geometric Series Sum Calculator

Scenario: 1

John is a mathematics student who is learning about geometric series. He is tasked with writing a program to calculate the sum of the first N terms of a specific geometric series. The series he needs to work with is $1 + 1/2 + 1/4 + 1/8 + \dots 1/N$, where each term is obtained by dividing the previous term by 2.

Task:

Craft a program that takes an input from the user and N representing the number of terms and that calculates the sum of the first N terms of the given geometric series. Finally, the program should output the sum of the first N terms of the series with **2 digit precision**. For negative input print **0.00** as output.

(The statements inside brackets are for your understanding purpose only)

Sample Input 1:

6 (Number of Terms in the series)

Sample Output 1:

1.97 (Sum of the Geometric series)

Sample Input 2:

-1

Sample Output 2:

0.00

Input	Output	Score
12	2.00	0
4	1.88	10
3	1.75	10
7	1.98	10
-7	0.00	10
6	1.97	10

9 Sum of Squares of N Natural Numbers

Scenario:

Developing a C program to calculate the sum of the squares of the first N natural numbers using nested while loops. This program should prompt the user to input the value of N and then iteratively compute the sum by squaring each natural number from 1 to N and adding them together.

Task:

Craft a program that prompts the user to input a positive integer N. Calculate the sum of the squares by squaring each natural number from 1 to N and adding them together.Ensure the program handles invalid inputs if N is less than or equal to 0 then Handle the input as “Invalid input”
(The statements inside brackets are for your understanding purpose only)

Sample Input 1:

5 (input)

Sample Output 1:

55 (sum of Squares till 5)

Sample Input 2

-10

Sample Output 2:

Invalid input

Input	Output	Score
50	42925	0
25	5525	10
12	650	10
21	3311	10
72	127020	10
-5	Invalid input	10

10 Harmonic Series

Scenario:

Design a program that ventures into the realm of infinite series, summing up the harmonic progression $1 + 1/2 + 1/3 + 1/4 + 1/5 + \dots$ up to 'n', where 'n' is a positive integer provided by the user.

Task:

Craft a program that takes the positive integer 'n' from the user and then meticulously computes the sum of the harmonic series up to 'n' . If a non-positive number is given, print "**Invalid Input**". Print the results **with two decimal points(precision 2)**.
(The statements inside brackets are for your understanding purpose only)

Sample Input : 1

10 (The value of N in the series)

Sample Output : 1

2.93

Sample Input : 2

-15

Sample Output : 2

Invalid Input

Input	Output	Score
4	2.08	10
5	2.28	0
24	3.78	10
61	4.70	10
35	4.15	10
-27	Invalid Input	10

11 Digits Count

Scenario:

Developing a program for a digital calculator company. As part of the new feature implementation, you're tasked with creating a code that counts the number of digits in an integer using a for loop. This feature will enhance the functionality of the calculator, allowing users to see the number of digits in their input.

Task:

Craft a program that takes an integer input from the user, counts the number of digits in the input using a loop, and then displays the count. Ensure that your program handles all the type of numbrs including negative numbers and zero for counting the digits.

(The statements inside brackets are for your understanding purpose only)

Sample Input : 1

47580 (input number)

Sample Output : 1

5 (number of digits)

Sample Input : 2

-234

Sample Output : 1

3

Input	Output	Score
78562453	8	0
4521	4	10
-2578963	7	10
0	1	10
-36528000	8	10
178596375	9	10

12 Square Pattern

Scenario:

In a bustling coding workshop, young programmer Alice is tasked with creating a square pattern using the '#' character. The project she's working on demands visually engaging representations, and this square pattern is just a small part of the larger scheme.

Task:

Craft a program that takes an input for the size of the square and prints a square pattern using the '#' character. Ensure the program validates input for positive integers only. Handle the numbers which are less than 1 as, "xx is Invalid input."

(The statements inside brackets are for your understanding purpose only)

Sample Input : 1

3 (Size of the rows and columns - square)

Sample Output : 1

#

Sample Input : 2

-10

Sample Output : 2

-10 is Invalid input.

Input	Output	Score
6	# #	10
5	# #	10
0	0 is Invalid input.	10
4	# # # # # # # # # # # # # # # #	0
-5	-5 is Invalid input.	10
2	# # # #	10

13 Pyramid Pattern

Scenario:

Imagine you are developing a program to generate a pyramid pattern with numbers increased by 1. Users will input the number of rows they want in the pyramid. The program will then display a triangular pattern where each row contains consecutive numbers starting from 1.

Task:

Craft a program that takes user input for the number of rows and generates a pyramid pattern with increasing numbers. The numbers should start from 1 and increment by 1 in each row. Handle the negative integer and 0 by printing “Invalid”

(The statements inside brackets are for your understanding purpose only)

Sample Input : 1

4 (Number of Rows)

Sample Output : 1

1
2 3
4 5 6
7 8 9 10

Sample Input : 2

-3

Sample Output : 2

Invalid

Input	Output	Score
3	1 2 3 4 5 6	10
2	1 2 3	10
0	Invalid	10
5	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	0
6	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21	10
-3	Invalid	10

14 Swap the Digits

Scenario:

Tom is an enthusiastic programmer, and was challenged by his friend Alice to program a program that swaps the first and last digits of a given number. Tom, always up for a coding challenge, decided to create a program that not only accomplishes the task but also ensures it handles various cases, such as numbers with less than two digits. With determination and a cup of coffee, Tom embarked on the coding journey.

Task:

Craft a program that prompts the user to input a number. Then it swaps the first and last digits of the given number.

(The statements inside brackets are for your understanding purpose only)

Sample Input : 1

12345 (input)

Sample Output : 1

52341 (After swapping first and last digits)

Sample Input : 2

798456

Sample Output : 2

698457

Input	Output	Score
258714	458712	10
428571	128574	10
571428	871425	10
714285	514287	0
857142	257148	10
109876	609871	10

15 Perfect Cubes

Scenario:

Imagine you are designing a program for a mathematical enthusiast who wants to explore the world of perfect cubes. The program takes user input and prints the cubes of the three consecutive numbers starting from the provided input. However, it also checks if the input is a positive integer, providing an "Invalid" message otherwise.

Task:

Craft a program that prompts the user to input a number, checks if it's positive, and if so, prints the cubes of the three consecutive numbers (starting from the given number) and Handle the Negative number and 0 by printing "Invalid".

(The statements inside brackets are for your understanding purpose only)

Sample Input : 1

10 (Input)

Sample Output : 1

1000 (Cube of 10)

1331 (Cube of 11)

1728 (Cube of 12)

Sample Input : 2

-5

Sample Output : 2

Invalid

Input	Output	Score
4	64 125 216	10
1	1 8 27	0
0	Invalid	10
7	343 512 729	10
-3	Invalid	10
15	3375 4096 4913	10

16 Roman numerals

Scenario:

You are developing a program that converts numbers to Roman numerals for a user interface. The program should take a positive integer as input and provide the corresponding Roman numeral output. Your goal is to implement and the user should be able to interactively input a number.

Task:

Craft a program that converts a given positive integer in a given range (1 to 40) and provides the corresponding Roman numeral output.If the given number is non positive number or greater than 40 then it should print as "Invalid input".

I - 1, IV - 4, V - 5, IX - 9, X -10, XL - 40 take as reference

(The statements inside the brackets are for understanding purpose only)

SAMPLE INPUT 1:

37 (Input)

SAMPLE OUTPUT 1:

XXXVII

SAMPLE INPUT 2:

56

SAMPLE OUTPUT 2:

Invalid input

Input	Output	Score
16	XVI	10
9	IX	0
40	XL	10
-7	Invalid input	10
12	XII	10
30	XXX	10

17 Sum of N odd and Even numbers

Scenario:

Tom is a computer science student who is learning C programming. His instructor has given him a task to write a program that displays the first n terms of odd and even natural numbers up to a given limit n and calculates their respective sums. Tom is excited to work on this task as it will help him reinforce his understanding of loops, conditionals, and basic arithmetic operations in C programming.

Task:

Craft a program that prompts the user to enter a value n and then displays the first n terms of odd and even natural numbers along with their sums.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

5 (input - number of terms starting from 1)

Sample Output 1:

9 (odd sum)

6 (even sum)

Sample Input 2:

2 (input)

Sample Output 1:

1 (odd sum)

2 (even sum)

Input	Output	Score
8	16 20	10
4	4 6	0
10	25 30	10
7	16 12	10
15	64 56	10
30	225 240	10

18 Detecting Narcissistic Numbers

Scenario :

In a small town, there was a young mathematician named Alice who loved to explore number patterns. One day, she stumbled upon the concept of narcissistic numbers - those magical integers where the sum of each digit raised to the power of the total number of digits equals the original number itself. Fascinated by this concept, Alice decided to create a program to identify narcissistic numbers.

Task:

Craft a program that prompts the user to input an integer, and counts the number of digits in the input number, calculates the sum of each digit raised to the power of the total number of digits, and finally checks if the result equals the original number. Print whether the number is narcissistic or not.

Hint:

$153 = 1^3 + 5^3 + 3^3$ where 3 in power is the number of digits in the given number

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

153 (Integer)

Sample Output 1:

153 is a narcissistic number

Sample Input 2:

126 (Integer)

Sample Output 2:

126 is not a narcissistic number

Input	Output	Score
0	0 is a narcissistic number	0
1	1 is a narcissistic number	10
370	370 is a narcissistic number	10
42	42 is not a narcissistic number	10
1634	1634 is a narcissistic number	10
12	12 is not a narcissistic number	10

19 Digit Sum Calculator

Tom, a budding programmer, is eager to learn more about coding. He recently discovered the joy of solving mathematical problems through programming. Today, he's exploring a program that calculates the sum of even and odd digits within an integer. With a keen interest in numbers, Tom dives into the code, eager to understand its workings and perhaps even improve upon it.

Task:

Craft a program that prompts the user to input an integer. It then proceeds to calculate the sum of even and odd digits within that integer using a while loop. Depending on whether the extracted digit is even or odd, it is added to the respective sum variables. Finally, the program prints out the sums of even and odd digits. Assume the number of digits may vary in size upto a maximum of 10 digits.

Hint:

Extract the each digit of the integer, and check whether the digit is even or odd.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

123456 (Enter an integer)

Sample Output 1:

12 (Sum of even digits)

9 (Sum of odd digits)

Sample Input 2:

5627

Sample Output 2:

8

12

Input	Output	Score
92438	14 12	0
86953	14 17	10
37159	0 25	10
45987	12 21	10
768543	18 15	10
529032959	4 40	10

20 Alphabet Triangle Generator

Scenario :

In a small village, there lived a group of children who loved playing with alphabet blocks. They were fascinated by the shapes they could create with these blocks, and one day, they decided to write a program that would generate a triangle made of alphabet characters.

Task:

Craft a program starts by taking input for the number of rows in the triangle, denoted by 'N'. Then, it iterates through each row, printing consecutive alphabet characters starting from 'A'. After printing each row, it moves to the next character and the next line until the triangle is complete for that test case. If the given input is less than or equal to zero, then print the output as "Invalid".

(The statements inside the brackets are for understanding purpose only)

Sample Input 1 :

3 (number of rows)

Sample Output 1 :

A
BB
CCC

Sample Input 2 :

0

Sample Output 2 :

Invalid

Input	Output	Score
2	A BB	0
3	A BB CCC	10
4	A BB CCC DDDD	10
-3	Invalid	10
4	A BB CCC DDDD	10
-6	Invalid	0
5	A BB CCC DDDD EEEE	10

21 Finding the Next Palindrome

SCENARIO:

Sarah is fascinated by numbers and patterns. She's particularly interested in palindromic numbers, those that read the same forwards and backwards. One day, she decided to write a program that would take an integer from the user and find the next palindrome greater than that integer. Excited to see how her program performs, she eagerly awaits the results.

Task:

Craft a program that prompts the user to enter an integer. Find the next palindrome number which is greater than the given integer. Print the next palindrome number as output.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

123 (input)

Sample Output 1:

131 (Next Palindrome number greater than input)

Sample Input 2:

4567

Sample Output 2:

4664

Input	Output	Score
1234	1331	0
5432	5445	10
999	1001	10
600	606	10
1120	1221	10
866	868	10

22 Finding Consecutive Palindromic Numbers

Scenario:

In a small town nestled between hills, there lived a curious mathematician named Alice. Alice was fascinated by palindromic numbers, those mysterious figures that read the same backward as forward. She decided to write a program that could generate palindromic numbers, starting from a given number. With determination in her heart and a cup of hot tea by her side, she delved into coding.

Task:

Craft a program that prompts the user to enter a starting number and then generates next five palindromic numbers one by one , using loops only.

Hint:

Focus on reversing the digits of a number and comparing it with the original number to determine if it's a palindrome.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

10

Sample Output 1:

11 22 33 44 55

Sample Input 2:

99

Sample Output 2:

99 101 111 121 131

Input	Output	Score
500	505 515 525 535 545	0
11	11 22 33 44 55	10
12345	12421 12521 12621 12721 12821	10
777	777 787 797 808 818	10
650	656 666 676 686 696	10
616	616 626 636 646 656	10

23 Generating Palindromic Right-Angled Triangle

Scenario :

John, a computer science student, is working on a project to generate special patterns using C programming. He has been tasked with creating a program that prints a palindromic right-angled triangle. Each row of this triangle consists of numbers in ascending order followed by the same numbers in descending order. John is excited to tackle this challenge and create a program that meets the requirements.

Task:

Craft a program that generates a palindromic right-angled triangle based on the input provided by the user. The program should take the number of rows as input and then print the triangle according to the specified pattern.It then iterates through each row, printing numbers in ascending order from 1 to the current row number.After printing the ascending numbers, it prints the descending numbers, forming a palindrome with the ascending numbers as the middle.The program continues this pattern for each row until the desired number of rows is reached. If the given input is less than or equal to zero, then print the out as "Invalid".

Hint:

This program will print a palindromic right-angled triangle where each row consists of numbers in ascending order followed by the same numbers in descending order.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

6
1
2 3 2
4 5 6 5 4
7 8 9 10 9 8 7
11 12 13 14 15 14 13 12 11
16 17 18 19 20 21 20 19 18 17 16

Sample Input 2:

-4
Sample Output 2:
Invalid

Input	Output	Score
4	1 2 3 2 4 5 6 5 4 7 8 9 10 9 8 7	0
2	1 2 3 2	10
3	1 2 3 2 4 5 6 5 4	10
0	Invalid	10
5	1 2 3 2 4 5 6 5 4 7 8 9 10 9 8 7 11 12 13 14 15 14 13 12 11	10
-2	Invalid	10

24 The Palindromic Sum

Scenario :

In a quaint town, there was a curious young programmer named Alice. One day, while browsing through old books in the town library, she stumbled upon a mysterious ancient script. It seemed to describe a ritual involving finding and summing up palindromic numbers. Fascinated by this discovery, Alice decided to translate the script into code and see what secrets it held.

Task:

Craft a program that prompts the user to input a positive integer. The program then iterates through numbers from 1 to the given number, checking if each number is a palindrome (reads the same forwards and backwards). If a number is a palindrome, it adds it to the sum. Finally, the program prints out the sum of all palindrome numbers less than or equal to the given positive integer.

Hint:

You'll need to reverse the digits of each number to check if it's a palindrome and sum all palindrome numbers upto given number as output.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

10

Sample Output 1:

45

Sample Input 2:

99

Sample Output 2:

540

Input	Output	Score
19	56	0
36	111	10
95	441	10
150	1145	10
70	276	10
170	1457	10

25 Pattern printing with multiples of 5

Scenario:

The program asks the user to enter the number of rows for the pattern (e.g., 4).The user enters the number of rows. The code uses a nested loop to iterate through rows and columns. Within the inner loop, it calculates values based on the current row (i) and multiplies by 5. This generates a sequence starting from 5 for each row. For each column iteration, the code prints the calculated value. After a complete row, it moves to the next line

Task:

- Create a program that gets the rows from the user
- Display “Invalid” if the rows are less than 1
- Print the number of rows with the multiples of 5
- It should increase as the row increases

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

6 (number of rows)

Sample Output1 :

5
5 10
5 10 15
5 10 15 20
5 10 15 20 25
5 10 15 20 25 30

Sample Input 2:

1 (number of rows)

Sample Output 2:

5

Input	Output	Score
-5	Invalid	0
5	5 5 10 5 10 15 5 10 15 20 5 10 15 20 25	10
3	5 5 10 5 10 15	10
7	5 5 10 5 10 15 5 10 15 20 5 10 15 20 25 5 10 15 20 25 30 5 10 15 20 25 30 35	10
-8	Invalid	10
4	5 5 10 5 10 15 5 10 15 20	10

26 Squares of Even Numbers & odd number Series

You've been assigned to develop a program that analyses sequences of numbers provided by the user. The program should prompt the user to enter a single integer. Depending on whether the input is even or odd, the program will generate a sequence of squares. For even input, it will print squares of even numbers up to the given input, while for odd input, it will print squares of odd numbers.

Task:

Craft a program that interacts with the user to obtain an integer input. Based on whether the input is even or odd, the program should generate a series of squares. If the input is even, the program should print squares of even numbers up to the entered number. If the input is odd, it should print squares of odd numbers up to the entered number.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

6 (input value)

Sample Output 1:

4 16 36

Sample Input 2:

11

Sample Output 2:

1 9 25 49 81 121

Input	Output	Score
7	1 9 25 49	0
12	4 16 36 64 100 144	10
9	1 9 25 49 81	10
21	1 9 25 49 81 121 169 225 289 361 441	10
8	4 16 36 64	10
16	4 16 36 64 100 144 196 256	10

27 Prime Pattern

You are tasked with creating a program that interacts with the user to generate a right-angled triangle. The program should prompt the user to input the number of rows for the triangle. Upon receiving the input, it will construct the triangle and identify prime-numbered rows. For each prime-numbered row, the program will print an asterisk (*), representing that row.

Task:

Craft a program prompts the user to enter the number of rows for the right-angled triangle. Then, it iterates over each row and checks whether the row number is prime or not. If it's prime, it prints '*' for that row.

(The statements inside the brackets are for understanding purpose only)

Note : 1 is not a Prime Number, and every asterisk(*) as a white space at last.

Sample Input 1:

6 (Input)

Sample Output 1 :

* *
* * *
* * * * *

Sample Input 2:

10

Sample Output 2 :

* *
* * *
* * * * *
* * * * * * *

Input	Output	Score
5	* * * * * * * * * *	0
8	* * * * * * * * * * * * * * * * *	10
2	* *	10
4	* * * * *	10
7	* * * * * * * * * * * * * * * * *	10
	* * * * *	

28 LCM Finder

You are a software developer working on a program that calculates the Least Common Multiple (LCM) of two numbers. The LCM is the smallest positive integer that is divisible by both numbers without leaving a remainder.

Task:

Craft a program to find the LCM of two given numbers. Your program should take user input for two positive integers, if any of the input is negative then print **Negative Input** else perform the LCM calculation, and then print the result.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

12 18 (two input values)

Sample Output 1:

36

Sample Input 2:

12 14

Sample Output 2:

84

Input	Output	Score
-1 -2	Negative Input	0
45 1	45	10
45 4	180	10
5 4	20	10
5 7	35	10
5 78	390	10

29 The Perfect Number Detective

In a quaint little town, there lived a curious detective named Sam. Sam wasn't your typical detective; instead of solving mysteries of thefts or murders, Sam was fascinated with numbers. One day, while sipping coffee at his favorite caf  , Sam overheard a conversation about perfect numbers – those magical digits whose divisors add up to the number itself. Intrigued, Sam decided to embark on a quest to create a program that could detect perfect numbers and unveil their largest divisor.

Task:

Craft a program prompts the user to input a number. Then iterates through all the numbers (from 1) less than the input number to find its divisors, add the divisors. If the sum is equal to the same number then it is perfect number. Check whether the given number is perfect or not and if the given number is perfect then find the largest divisor of the given number else print the given number as it is and print the output

Hint:

Consider using loops to iterate through all possible divisors of the given number and keep track of the sum of divisors and the largest divisor found.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

6

Sample Output 1:

3

Explanation:

Divisors of 6 are 1,2 and 3. The sum of 1,2, and 3 is 6 which is equal to the given number 6. So, it is perfect number. So, 3 is the largest divisor.

Sample Input 2:

10

Sample Output 2:

10

Explanation:

Divisors of 10 are 1,2 and 5. The sum of 1,2, and 5 is 8 which is not equal to the given number 10. So, it is not perfect number. So, print the same number 10 as output.

Input	Output	Score
28	14	0
12	12	0
496	248	10
50	50	10
8128	4064	10
33550336	16775168	10
501	501	10

30 Handshake Simulation Program

In a bustling conference room, attendees are arriving for a networking event. As they gather, the organizer, eager to understand the potential connections being made, decides to develop a program to simulate the number of handshakes that will occur. With this program, they can estimate the networking potential and the overall interaction among attendees.

Task:

Craft a program that prompts the user to input the total number of people attending an event. The program should then simulate handshakes between each pair of people and Count the total number of handshakes that occur. Print out the total number of handshakes.

Hint:

Think about how you can iterate through each person and pair them with every other person to simulate handshakes.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

6 (Input)

Sample Output 1:

15 (total number of handshakes)

Sample Input 2:

10 (Input)

Sample Output 2:

45 (total number of handshakes)

Input	Output	Score
3	3	0
5	10	0
8	28	10
22	231	10
15	105	10
20	190	10
45	990	10

31 Odd or Even numbers series

As a teacher in a primary school, you've been tasked with creating a program to help students understand the concept of even and odd numbers. You want to develop a program that takes a user-defined limit as input and then prints out the series of even or odd numbers within that limit.

Task:

Craft a program first prompts the user to enter a limit. Then it checks whether the limit is even or odd using the modulo operator (%). After that, it uses loops to print out the series of even or odd numbers up to the limit, depending on the input provided by the user. If the limit is less than or equal to 0 then print **Not a Positive integer**

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

9 (limit)

Sample Output 1:

1 3 5 7 9 (Odd numbers)

Sample Input 2:

4

Sample Output 2:

2 4

Input	Output	Score
-4	Not a Positive integer	0
7	1 3 5 7	10
8	2 4 6 8	10
9	1 3 5 7 9	10
15	1 3 5 7 9 11 13 15	10
18	2 4 6 8 10 12 14 16 18	10

32 Digit Summation

In a quaint village nestled between rolling hills, there lived a curious mathematician named Milo. Milo was fascinated by the properties of numbers and spent his days concocting various mathematical puzzles. One day, he stumbled upon an ancient manuscript that spoke of a mystical ritual involving the summing of digits until only a single digit remained. Intrigued, Milo set out to decipher this enigma and embarked on a journey to uncover the secrets hidden within the digits.

Task:
Craft a program should prompt the user to input an integer, then repeatedly sum its digits until the sum becomes a single digit. Finally, it should output this single digit sum.

Hint:
Focus on how the code iteratively sums the digits of the input number until it becomes a single digit.

(The statements inside the brackets are for understanding purpose only)

Sample Input:

9876 (Input)

Sample Output:

3 (Sum of digits)

Explanation:

$9+8+7+6 = 30 = 3+0 =3$

Sample Input:

12345

Sample Output:

6

Input	Output	Score
70	7	0
998	8	0
631	1	10
440	8	10
913	4	10
66	3	10
8456	5	10

33 Vowel Counter

You are a software developer tasked with creating a simple program to count the number of vowels in a given input word. Your task is to design and develop the program according to the provided specifications, ensuring it accurately counts the vowels in the input word and provides the correct output.

Task:

Craft a program that counts the number of vowels in a user-provided input. The program should prompt the user to enter a word, read the input character by character, and increment a counter for each vowel encountered. After reading the entire input, it should display the total count of vowels found.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

Bestfriends

Sample Output 1:

3

Sample Input 2:

Heelloo

Sample Output 2:

4

Input	Output	Score
success	2	0
PersonalizedSkill	6	10
PSTeam	2	10
PSHeadquarters	5	10
helloworld	3	10
waterbottle	4	10

34 Digit Incrementer

In a futuristic world where encryption algorithms play a crucial role in securing data, a group of novice programmers are tasked with developing a simple yet effective encryption tool. The team, consisting of enthusiastic individuals from diverse backgrounds, strives to create an algorithm that can transform ordinary numbers into seemingly random sequences. As they brainstorm ideas, one team member suggests a concept where each digit of the input number is incremented by 1. With this concept in mind, they set out to implement the algorithm and test its effectiveness.

Task:
Craft a program that takes an integer input from the user. Store the original number and iterate through each digit of the number. For each digit, add 1 and store the modified digit (if the digit is 9, it becomes 0). Accumulate the modified digits to form the result. Output the modified numbers.

Hint:
Increment each digit of the input number by 1.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:
245
Sample Output 1:
356 (Modified number)
Sample Input 2:
9876
Sample Output 2:
987

Input	Output	Score
7345	8456	0
12589	23690	0
9765	876	10
3087	4198	10
51276	62387	10
8473	9584	10
20196	31207	10

35 Printing Pattern in Reverse Order

You're a programmer tasked with creating a program to generate a reverse pattern of numbers. This program will be part of a larger system used in educational environments to teach programming concepts. Your program should take user input for the number of rows and then output a pattern of numbers in reverse order.

Task:

Craft a program that prompts the user to input the number of rows for the pattern. To generate a pattern of numbers in reverse order. Each row should display numbers starting from the row number down to 1. If the input number is less than or equal to zero then print **Invalid Input**

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

4 (input no of rows)

Sample Output 1:

1 2 3 4
1 2 3
1 2
1

Sample Input 2:

3

Sample Output 2:

1 2 3
1 2
1

Note : There is a white space at end of every number and no new line is present after the last row.

Input	Output	Score
-74	Invalid Input	0
7	1 2 3 4 5 6 7 1 2 3 4 5 6 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1	10
8	1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 1 2 3 4 5 6 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1	10
2	1 2 1	10
1	1	10
000	Invalid Input	10

36 The Odd Factorial Quest

In a distant land, there lies a mystical village where the inhabitants are fond of odd numbers and the magic they hold. Legend has it that the key to unlocking the village's greatest treasure is hidden within the factorial of an odd number. Brave adventurers from far and wide journey to this village in hopes of discovering the secret. However, the village elders have devised a challenge: adventurers must input an odd number, and using their wits and basic programming skills, they must calculate its factorial without any modern conveniences. Will you be the one to uncover the treasure?

Task:

Craft a program prompts the user to enter an odd number.It checks if the entered number is odd and greater than zero. If not, it prompts the user to enter a valid odd number.Once a valid odd number is entered, the program calculates its factorial using a loop.The factorial is calculated by multiplying all odd numbers from 1 up to the entered odd number. Finally, the program prints the factorial of the input number. If the given input is even then print as "Please enter a valid odd number" otherwise whether the input is less than 1,then print as "Invalid".

Hint:

Utilize a loop to multiply all odd numbers up to the given odd number to calculate its factorial.

(The statements inside the brackets are for understanding purpose only)

Sample Input:

5 (Input)

Sample Output:

15 (Factorial of 5)

EXPLANATION

calculates the factorial of 5 (1 * 3 * 5) = 15

Sample Input:

6

Sample Output:

Please enter a valid odd number

Input	Output	Score
7	105	0
-3	Invalid	0
4	Please enter a valid odd number	10
40	Please enter a valid odd number	0
11	10395	10
13	135135	10
21	864408687	10
3	3	10

37 Series Expansion

Scenario :

In a mathematics workshop, students are exploring series expansion techniques. As part of the session, they are intrigued by a particular series: $1 - (x^2)/2! + (x^4)/4! - (x^6)/6! + \dots$. They want to develop a program to find the sum of this series.

Task :

The students are tasked with Craft, a program that calculates the sum of the series $1 - (x^2)/2! + (x^4)/4! - (x^6)/6! + \dots$. The program should prompt the user to input the value of 'x' and the number of terms 'n' in the series.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

7 (Value of X)
5 (Number of Terms)

Sample Output : 1

56.12 (Sum of the series)

Sample Input : 2

4 (Value of X)
7 (Number of Terms)

Sample Output: 2

-0.65 (Sum of the series)

Input	Output	Score
6 4	-27.80	10
9 5	563.39	10
10 6	-1296.80	0
7 7	7.17	10
9 8	-70.27	10
11 11	59.09	10

38 Fibonacci Even Number Generator

Jack, an amateur programmer, is fascinated by the Fibonacci sequence and its properties. He wants to create a program that generates even Fibonacci numbers up to a given limit. Inspired by his curiosity, he starts working on a simple C program to achieve this goal. With his determination and a few cups of coffee, Jack dives into coding.

Task:
Craft a program that prompts the user to enter a limit, then generates Fibonacci numbers up to that limit and prints only the even numbers from the series. For zero and negative number print "Invalid".

Hint:
Use a loop to generate Fibonacci numbers and check each number if it's even before printing.

Sample Input 1:
5
Sample Output 1:
0 2
Sample Input 2:
-2
Sample Output 2:
Invalid

Input	Output	Score
20	0 2 8	0
40	0 2 8 34	10
-6	Invalid	0
150	0 2 8 34 144	10
-2	Invalid	10
2600	0 2 8 34 144 610 2584	10
11000	0 2 8 34 144 610 2584 10946	10

39 Exploring the Growth Series

In a quaint little village nestled between rolling hills, there lived a curious young mathematician named Lily. Lily had always been fascinated by patterns and sequences. One day, while exploring the library in search of adventure, she stumbled upon an ancient scroll that described a mysterious series where each term was twice the previous one. Intrigued, Lily decided to decipher this series using her mathematical prowess.

Task:

Craft a program that takes input from the user for the number of terms in the series and the initial term. The program should then calculate and print the subsequent terms of the series, where each term is obtained by multiplying the previous term by 2.Finally, the program prints the output.

Hint:

The key to this task lies in understanding how to generate the subsequent terms of the series by multiplying each term by 2.

(The statements inside the brackets are for understanding purpose only)

Sample Input 1:

5 (number of terms)
3 (series starting value)

Sample Output 1:

3 6 12 24 48

Sample Input 2:

4
2

Sample Output 2:

2 4 8 16

Input	Output	Score
6 11	11 22 44 88 176 352	0
3 15	15 30 60	0
8 10	10 20 40 80 160 320 640 1280	10
1 7	7	10
7 3	3 6 12 24 48 96 192	10
5 36	36 72 144 288 576	10
10 4	4 8 16 32 64 128 256 512 1024 2048	10

40 Sum of all Prime Factors

Imagine you're working on a program to analyze the financial health of a company. As part of this analysis, you need to calculate the sum of all prime factors of various financial metrics such as revenue, expenses, and profits.

Task:

Craft a program that prompts the user to input an integer. Find the prime factors of the given number and display their sum.

Hint : A number's factors which are prime numbers are prime factors

(The statements inside the brackets are for understanding purpose only)

Sample Input : 1

36 (Number)

Sample Output : 1

10 (Sum of Prime Factors)

Sample Input : 2

52 (Number)

Sample Output : 2

17 (Sum of Prime Factors)

Input	Output	Score
28	11	0
85	22	0
102	22	10
46	25	10
160	15	10
78	18	10
22	13	10

41 Sum N Odd Natural Numbers

You work for a community centre that offers various educational programs for children. As part of the upcoming maths enrichment classes, you're tasked with creating a program to help students understand odd natural numbers better. The program should display the first 'n' odd natural numbers and calculate their sum.

Task:

Craft a program that prompts the user to input the value of 'n', representing the number of odd natural numbers they want to display. Then, the program should output the first 'n' odd natural numbers along with their sum.

(The statements inside the brackets are for understanding purpose only)

Sample Input : 1

5 (Value of N)

Sample Output : 1

1 3 5 7 9 (Natural numbers)

25 (Sum of 5 odd natural numbers)

Sample Input : 2

7 (Value of N)

Sample Output : 2

1 3 5 7 9 11 13 (Natural numbers)

49 (Sum of 7 odd natural numbers)

Input	Output	Score
4	1 3 5 7 16	0
6	1 3 5 7 9 11 36	0
8	1 3 5 7 9 11 13 15 64	10
3	1 3 5 9	10
9	1 3 5 7 9 11 13 15 17 81	10
5	1 3 5 7 9 25	10
10	1 3 5 7 9 11 13 15 17 19 100	10

42 Check Second Even Number

Scenario :

In a data analysis project, you're tasked with developing a program to determine if a user-defined range is even. If the range is indeed even, the program must then identify and output the second even number within that range.

Task:

Craft a program that prompts the user to input a range. Upon verification that the range is even, utilize loops to identify and print the second even number within the specified range.

(The statements inside the brackets are for understanding purpose only)

Sample Input : 1

14 20 (Starting & Ending number)

Sample Output : 1

16 (Second even number)

Sample Input : 2

15 41 (Starting & Ending number)

Sample Output : 2

15 and 41 are not Even.

Input	Output	Score
16 40	18	0
80 120	82	10
75 25	75 and 25 are not Even.	10
150 500	152	10
79 45	79 and 45 are not Even.	10
60 70	62	10

43 Floyd's Triangle

Scenario :

Imagine you are tasked with creating a program to generate Floyd's Triangle, a unique numerical pattern. Floyd's Triangle is a right-angled triangular sequence of natural numbers. The task is to write a program that takes user input for the number of rows and prints Floyd's Triangle accordingly.

Task:

Craft a program that prompts the user to enter the number of rows for Floyd's Triangle. The program should then generate and display Floyd's Triangle, starting from 1 and incrementing consecutively in each row. Handle the Negative number us (â€œInvalidâ€) (The statements inside the brackets are for understanding purpose only)

Sample Input : 1

4 (Rows for Floyd)

Sample Output : 1

1
2 3
4 5 6
7 8 9 10

Sample Input : 2

-2 (Rows for Floyd)

Sample Output : 2

Invalid

Input	Output	Score
4	1 2 3 4 5 6 7 8 9 10	0
5	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	10
2	1 2 3	10
-2	Invalid	10
-7	Invalid	10
8	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36	10

44 Automorphic Number

Scenario :

As the mathematics club gathered, they set up a challenge to find Automorphic Numbers. Curious students eagerly examined their chosen digits, hoping to discover the magical numbers where the square of the number ends with the same digits. A triumphant cheer echoed as participants successfully identified these intriguing Automorphic Numbers.

Task:

Given an integer input as the number, the objective is to check whether or not the number is an automorphic number. First, square the input number and check if the number has been repeated in the same order of digits at the end of its square, whether it is an automorphic number or not.

Example:

- 25 is an automorphic number because $25^2 = 625$, and the last two digits of 625 are 25.
- 23 is not an automorphic number because $23^2 = 529$, and the last two digits are not 23.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

6 (Number)

Sample Output : 1

6 is an Automorphic Number.

Sample Input : 2

7 (Number)

Sample Output : 2

7 is not an Automorphic Number.

Input	Output	Score
25	25 is an Automorphic Number.	0
15	15 is not an Automorphic Number.	10
8	8 is not an Automorphic Number.	10
76	76 is an Automorphic Number.	10
376	376 is an Automorphic Number.	10
5	5 is an Automorphic Number.	10

45 Sum of First and Last digit

Scenario :

Imagine you're developing a program for a basic arithmetic calculator. One of the requirements is to calculate the sum of the first and last digits of a given number. This feature is crucial for users who need to quickly verify the integrity of long numerical inputs.

Task:

Craft a program that prompts the user to input a number. The program then extracts the first and last digits of the input number and calculates their sum.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

12345 (Value)

Sample Output : 1

6 (Sum of first and last digit)

Sample Input : 2

78956

Sample Output : 2

13 (Sum of first and last digit)

Input	Output	Score
785926	13	0
45126	10	10
98745	14	10
7562	9	10
9999	18	10
45623	7	10

46 Sum of Even Numbers

Scenario :

Program prompts users for a positive integer 'N', then calculates and displays the sum of even numbers from 1 to N. It validates user input for positivity, utilizing a loop that increments by 2 to cover all even integers in the range. The cumulative sum is calculated and promptly presented, ensuring a user-friendly and efficient experience. The Output reflects the sum of even numbers within the specified range.

Tasks :

- 1. User Input: Prompt the user to enter a positive integer N.
- 2. Calculate Sum: Use a loop to iterate through all even numbers between 1 and N and calculate their sum.
- 3. Display Result: Print the calculated sum.

Example : If N = 6, the program would calculate the sum of even numbers (2 + 4 + 6) and output: 12

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

20 (Integer)

Sample Output : 1

110 (Sum of even numbers)

Sample Input : 2

-12 (Integer)

Sample Output : 2

-12 is an Invalid number.

Input	Output	Score
11	30	0
25	156	10
31	240	10
-6	-6 is an Invalid number.	10
40	420	10
-10	-10 is an Invalid number.	10

47 XOR Operations

Scenario :

You are a software engineer working on a cutting-edge encryption algorithm that utilizes bitwise XOR operations to secure sensitive information. As part of the development process, you need to create a program that performs a specific operation on a given number using only the bitwise XOR operator (^) without using strings or arrays.

Task :

Craft a program that takes an integer as input and performs the following operation using only the bitwise XOR operator:

- If the input number is even, divide it by 2.
- If the input number is odd, multiply it by 3 and add 1.
- Continue this process until the resulting number is 1. Count and return the number of steps taken to reach 1.

Example :

6 is even, so we divide by 2:
 $6 \div 2 = 3$
3 is odd, so we multiply by 3 and add 1:
 $3 \times 3 + 1 = 10$
10 is even, so we divide by 2:
 $10 \div 2 = 5$
5 is odd, so we multiply by 3 and add 1:
 $5 \times 3 + 1 = 16$
16 is even, so we divide by 2:
 $16 \div 2 = 8$
8 is even, so we divide by 2:
 $8 \div 2 = 4$
4 is even, so we divide by 2:
 $4 \div 2 = 2$
2 is even, so we divide by 2:
 $2 \div 2 = 1$

And again, the process stops here because we've reached 1.
So, for the input of 6, the number of steps taken is 8.
(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

6 (Number)

Sample Output : 1

8 (Number of steps taken)

Sample Input : 2

25 (Number)

Sample Output : 2

23 (Number of steps taken)

Input	Output	Score
16	4	0
30	18	10
46	16	10
72	22	10
7	16	10

48 Sum of the Middle Digits

Scenario :

In a banking application, there's a need to compute the sum of the middle digits of a customer's account number, excluding the first and last digits. This ensures a layer of security by validating account information without exposing sensitive data.

Task :

Craft a program to calculate the sum of the middle digits of a given account number, provided by the user. Utilize loop concepts to iterate through the digits, excluding the first and last, then sum them up.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

14562 (Value)

Sample Output : 1

15 (Sum of middle digits)

Sample Input : 2

10254823 (Value)

Sample Output : 2

21 (Sum of middle digits)

Input	Output	Score
369852	28	0
4789562	35	10
789	8	10
8523	7	10
365214	14	10
87456	16	10

49 Reverse the digits

Scenario :

You are a software developer working on a program designed to assist with numerical manipulations. Your current task involves creating a program that reverses a positive number entered by the user. The program should not only prompt the user for a positive number but also ensure the validity of the input. Once a valid input is received, the program should use loops and basic arithmetic operations to reverse the order of the integer's digits.

Task:

Create a program that takes a positive number as input and prints its reverse. Ensure the program prompts the user for input, validates that the entered number is positive, and uses loops to reverse the order of its digits. If it is a negative number, then print (**x is not a Positive Number.**)
(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

96387 (Input)

Sample Output : 1

78369 (Output)

Sample Input : 2

00710.823 (Input)

Sample Output : 2

17 (Output)

Input	Output	Score
000123	321	0
9963821	1283699	10
5869	9685	10
-52	-52 is not a Positive Number.	10
78.36921	87	10
29876.235	67892	10

50 Perfect Square

Scenario :

As a prospective developer, you have to create a software that checks to see if a given positive number is a perfect square. A perfect square in mathematics is a number that may be expressed as the result of multiplying a number by itself. A number that may be written as the product of a number and itself is known as a perfect square.

Task :

Craft a program that prompts users for a positive integer, checks if it's a perfect square using loops and conditionals. (Negative value should be taken as Invalid Number.)
(The statements inside the brackets are for understanding purpose only.)

Sample Input: 1

16.00 (Input)

Sample Output : 1

16 is a Perfect Square.

Sample Input : 2

02.53 (Input)

Sample Output : 2

2 is not a Perfect Square.

Input	Output	Score
-26.399	-26 is Invalid Number.	0
400.5	400 is a Perfect Square.	10
25.00	25 is a Perfect Square.	10
785	785 is not a Perfect Square.	10
-1	-1 is Invalid Number.	10
9	9 is a Perfect Square.	10

51 Alphabetical Pattern

Scenario :

The program prompts the user to enter the number of rows for the pattern. The user enters a valid number, say 4, indicating the desired number of rows. The code validates the user input (ensuring it's between 1 and 26). Since the input is valid, the code enters a loop to print the pattern. The pattern is printed with 4 rows, where each row displays characters in alphabetical order (A, AB, ABC, ABCD).

Task :

Craft a program that gets the number of rows as input from the user and prints the alphabetic pattern. The program should print "XY is **Invalid input**." if the number of rows is less than 0 or above 26 since there are only 26 alphabets.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

5 (Number of Rows)

Sample Output : 1

A
AB
ABC
ABCD
ABCDE

Sample Input : 2

30 (Number of Rows)

Sample Output : 2

30 is an Invalid input.

Input	Output	Score
4	A AB ABC ABCD	10
3	A AB ABC	0
6	A AB ABC ABCD ABCDE ABCDEF	10
-10	-10 is an Invalid input.	10
27	27 is an Invalid input.	10
7	A AB ABC ABCD ABCDE ABCDEF ABCDEFG	10

52 Multiples of 5 - Pattern Printing

Scenario :

The program asks the user to enter the number of rows for the pattern. The user enters the number of rows. The code uses a nested loop to iterate through rows and columns. Within the inner loop, it calculates values based on the current row (i) and multiplies by 5. This generates a sequence starting from 5 for each row.

Task :

- Craft a program that gets the rows from the user.
- Display "XY is an Invalid input" if the rows are less than 1.
- Print the number of rows with the multiples of 5.
- It should increase as the row increases.

(The statements inside the brackets are for understanding purpose only)

Sample Input : 1

6 (Number of Rows)

Sample Output : 1

5
5 10
5 10 15
5 10 15 20
5 10 15 20 25
5 10 15 20 25 30

Sample Input : 2

1 (Number of Rows)

Sample Output : 2

5

Input	Output	Score
3	5 5 10 5 10 15	0
-35	-35 is an Invalid input.	10
8	5 5 10 5 10 15 5 10 15 20 5 10 15 20 25 5 10 15 20 25 30 5 10 15 20 25 30 35 5 10 15 20 25 30 35 40	10
6	5 5 10 5 10 15 5 10 15 20 5 10 15 20 25 5 10 15 20 25 30	10
0	0 is an Invalid input.	10
5	5 5 10 5 10 15 5 10 15 20 5 10 15 20 25	10

53 Product of N digits

Scenario :

Imagine a cutting-edge device that multiplies the digit of any number it encounters. Users can input a number, select a digit in the number, and witness the product of those digits in the input number. Whether for mathematical exploration or educational purposes, this innovative product brings a new dimension to understanding the significance of individual digits in numbers.

Task :

Craft a program to input as an integer or string, use a loop to calculate the product of n digits in given the number using a while loop if the output is zero, then print **Invalid input.**
(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

1258 (Value)

Sample Output : 1

80 (Output Product)

Sample Input : 2

12508 (Input Value)

Sample Output : 2

Invalid Input. (Output)

Input	Output	Score
741258	2240	0
453789	30240	10
928679861	2612736	10
6782341	8064	10
35689	6480	10
998045	Invalid Input.	10

54 Prime Number in range

Scenario :

In a small village fair, children participate in a game where they select a number of two different ranges. The game master, with enthusiasm, announces that a special prize awaits those who choose a prime number. Excitement fills the air as the children eagerly search for their lucky prime picks.

Task:

Craft a program that takes two integers as input. To calculate the prime number within the specified range of two integers. Print the output by using the loops.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

10 (Starting Range)

25 (Ending Range)

Sample Output : 1

11 13 17 19 23 (Output)

Sample Input : 2

2 (Starting Range)

5 (Ending Range)

Sample Output : 2

2 3 5 (Output)

Input	Output	Score
10 25	11 13 17 19 23	0
5 20	5 7 11 13 17 19	10
10 30	11 13 17 19 23 29	10
11 34	11 13 17 19 23 29 31	10
1 10	2 3 5 7	10
67 95	67 71 73 79 83 89	10

55 Harshad Number

Scenario :

In a maths class, students are exploring Harshad numbers. The teacher challenges them to find Harshad numbers. As the students enthusiastically calculate, they discover the intriguing properties of these divisible-by-sum-of-digits numbers.

Task :

Given an integer number as the input, the objective is to check whether or not the number is a Harshad Number by checking if the number is divisible by the sum of its digits or not(i.e is Not Harshad number or is Harshad number).

Example :

- The input number is 18.
- The sum of its digits is $1 + 8 = 9$.
- Now, we check if 18 is divisible by 9.
- Since 18 is divisible by 9 ($18 / 9 = 2$), it satisfies the condition.
- So, the output will be "18 is Harshad number."

(The statements inside the brackets are for understanding purpose only)

Sample Input : 1

18 (Input)

Sample Output : 1

18 is Harshad number. (Output)

Sample Input : 2

99 (Input)

Sample Output : 2

99 is Not Harshad number. (Output)

Input	Output	Score
24	24 is Harshad number.	0
25	25 is Not Harshad number.	10
12	12 is Harshad number.	10
36	36 is Harshad number.	10
51	51 is Not Harshad number.	10
21	21 is Harshad number.	10

56 Strong Number

Scenario:

In a small village, a curious mathematician discovers a mysterious pattern among certain numbers. With keen observation, they realise that a particular number's sum of its factorial digits equals the number itself, defining it as a strong number. As the villagers gather around, fascinated by the revelation, the mathematician demonstrates with examples, unravelling the secrets of these intriguing digits.

Task:

Craft a program to get the input as an integer. To split the number into digits and then the factorial of all digits. After finding the factorial of a digit, sum all the factorials. Then check if the sum of the factorial of the digit is equal to the input. If it is equal, then print it as "X is a Strong Number." If it is not equal, then print "X is Not a Strong Number".

(Hint: $123 = 1! + 2! + 3! = 1+2+6=9$, which is not equal to 123. So it is not a strong number.

(The statements inside the brackets are for understanding purpose only.)

Sample Input : 1

1 (Input)

Sample Output : 1

1 is a Strong Number.

Sample Input : 2

4534 (Input)

Sample Output : 2

4534 is Not a Strong Number.

Input	Output	Score
2	2 is a Strong Number.	0
345	345 is Not a Strong Number.	10
2567	2567 is Not a Strong Number.	10
145	145 is a Strong Number.	10
45908	45908 is Not a Strong Number.	10
40585	40585 is a Strong Number.	10

57 Palindrome Check

Scenario :

At the math club meeting, students engage in a friendly competition to find palindromic numbers. As they eagerly input their chosen digits, the club president challenges them to identify palindromes, adding an extra twist to the mathematical fun. Laughter and camaraderie fill the room as they collectively explore the world of numerical symmetry.

Task :

Craft a program to take input as numbers. Check if the input is a palindrome or not and print the output using the loops. If input is negative, then convert to positive, and then find Palindrome or not. **(The statements inside the brackets are for understanding purpose only.)**

Sample Input : 1

33233 (Input)

Sample Output : 1

33233 is Palindrome. (Output)

Sample Input : 2

3214 (Input)

Sample Output : 2

3214 is Not a Palindrome. (Output)

Input	Output	Score
569	569 is Not a Palindrome.	0
121	121 is a Palindrome.	10
66	66 is a Palindrome.	10
145	145 is Not a Palindrome.	10
77	77 is a Palindrome.	10
1064	1064 is Not a Palindrome.	10

